

Auditoria de Segurança — LexOffice

Escopo: repositório guimunhoz13/TCCrepository (backend Django/DRF + frontend Next.js), branch claude/site-styling-features-04vpvx.

Data: 10/09/2026

Metodologia: varredura forense do histórico completo do Git (FASE 1), revisão manual de código contra OWASP Top 10 2021 + auditoria automatizada de dependências com pip-audit e npm audit (FASE 2), correção do que era corrigível sem quebrar regras de negócio (FASE 3), e este relatório (FASE 4).

1. Resumo Executivo

Segredos vazados no histórico do Git	Nenhum encontrado (varredura completa, não só do estado atual)
Vulnerabilidades críticas encontradas	1 (chave de assinatura JWT fixa no código-fonte)
Vulnerabilidades altas encontradas	2 (upload de arquivo sem validação de tipo; exposição de mídia sem autenticação)
Vulnerabilidades médias/baixas encontradas	2 (ausência de rate limiting; .gitignore incompleto)
Dependências vulneráveis (backend)	3 pacotes, 8 CVEs — corrigido
Dependências vulneráveis (frontend)	9 vulnerabilidades, incluindo RCE não-autenticado — corrigido
Pontos fortes confirmados	Isolamento multi-tenant (BOLA/IDOR) correto em 100% dos endpoints revisados; CORS restrito (allowlist, não wildcard); zero uso de SQL bruto/eval/exec/pickle

Classificação de maturidade de segurança:

- **Antes da auditoria:** Risco **Alto** — uma única vulnerabilidade (SECRET_KEY fixa) permitiria a qualquer pessoa com acesso ao repositório (inclusive um ex-colaborador, ou qualquer pessoa que veja o GitHub público) forjar um token de autenticação válido para **qualquer escritório de advocacia cadastrado no sistema**, sem precisar de senha.
- **Depois das correções desta auditoria:** Risco **Baixo-Médio** — a vulnerabilidade crítica foi eliminada; restam duas recomendações de melhoria (proxy de download autenticado para documentos; rate limiting no login) que exigem mudanças arquiteturais maiores e foram documentadas para implementação futura, não aplicadas nesta passada para não arriscar quebrar o sistema às pressas.

2. FASE 1 — Varredura Forense do Histórico do Git

Foi feita uma busca no **histórico completo** (git log --all -p, git log --all --diff-filter=A --name-only), não apenas no estado atual dos arquivos, por:

- Chaves privadas (BEGIN RSA/DSA/EC/OPENSSH PRIVATE KEY)
- Tokens de nuvem/API (AWS AKIA. . . , GitHub ghp_. . . , Slack xox. . . , Google AIza. . . , OpenAI sk-. . .)
- Padrões password=/senha=/DB_PASSWORD= seguidos de valor não-óbvio
- Arquivos de nome sensível já commitados em qualquer momento da história: .env, .pem, .key, .p12, .pfx, .sql, .dump, .bak, .sqlite3, id_rsa, credentials.json
- IPs internos (faixas RFC1918) e e-mails pessoais em arquivos versionados

Resultado: Nenhum segredo real foi encontrado em nenhum momento do histórico.

Pontos verificados especificamente:

- backend/.env **nunca** foi commitado (confirmado via `git log --all --full-history -- backend/.env`, retorno vazio).
- backend/.env.example sempre conteve apenas placeholders (xxxxxxxxxxxxxxxxxxxxxx) em toda sua história — nunca teve a senha real do Supabase.
- As credenciais reais do projeto Supabase usado durante o desenvolvimento não aparecem em nenhum commit.
- Nenhuma chave privada, token de API de nuvem, ou credencial de terceiros foi encontrada.

Único achado real: arquivo de mídia de teste versionado

- **Arquivo:** backend/media/documentos/spidermanwpp.jpg
 - **Origem:** commit d4d5000 (upload de teste feito durante o desenvolvimento)
 - **Sensibilidade:** baixa — é uma imagem de teste, não um documento de cliente real
 - **Risco do padrão:** o diretório backend/media/ nunca esteve no .gitignore, o que significa que, se o sistema fosse usado com dados reais antes desta correção, **documentos de identidade de clientes (RG/CPF escaneados) entrariam permanentemente no histórico do Git** — inclusive se depois fossem apagados do disco, continuariam recuperáveis via `git log`.
 - **Correção aplicada:** arquivo removido do índice do Git (`git rm --cached`) e backend/media/ adicionado ao .gitignore (ver seção 4).
-

3. FASE 2 — Tabela de Vulnerabilidades (OWASP Top 10 / CWE)

#	Severidade	Vulnerabilidade	Local (arquivo:linha)	OWASP / CWE	Status
1	Crítica	SECRET_KEY do Django fixa como string literal no código-fonte. Como SIMPLE_JWT nunca era configurado, djangoestframework_simplejwt usa SECRET_KEY como SIGNING_KEY por padrão — ou seja, a mesma chave publicamente visível no histórico do GitHub assina os tokens JWT customizados (user_id, tipo_usuario, escritorio_id). Qualquer pessoa com acesso ao repositório podia forjar um JWT válido para qualquer escritório e com papel de administrador , sem senha, obtendo acesso total a todos os dados de todos os clientes do sistema.	backend/core/settings.py:19 (antes da correção)	A02:2021 Cryptographic Failures / A07:2021 Identification & Authentication Failures — CWE-798 (Use of Hard-coded Credentials), CWE-321 (Use of Hard-coded Cryptographic Key)	■ Corrigido

#	Severidade	Vulnerabilidade	Local (arquivo:linha)	OWASP / CWE	Status
2	Alta	Upload de "documento de identidade" e de documentos de processo aceitava qualquer extensão de arquivo, incluindo .html/.svg/.js. Combinado com o item 3 (mídia servida sem autenticação), um arquivo .svg com <script> embutido, aberto direto pela URL de mídia, executa no contexto de origem da aplicação (XSS armazenado).	backend/advocacia/models.py (campos do documento_identidade ×2, Documento.arquivo); serializers explícitos em backend/advocacia/serializers.py:166-167 (AdvogadoRegistroSerializer) e :250-254 (AdvogadoSerializer)	A03:2021 Injection (XSS) / A04:2021 Insecure Design — CWE-434 (Unrestricted Upload of File with Dangerous Type)	■ Corrigido
3	Alta	Em modo DEBUG=True, todos os arquivos enviados (fotos e documentos de identidade de clientes/advogados, documentos de processo) são servidos por django.views.static.serve sem nenhuma verificação de autenticação/autorização — basta conhecer (ou adivinhar) a URL. Como o FileField do Django mantém o nome original do arquivo, os nomes são previsíveis.	backend/core/urls.py:37-38	A01:2021 Broken Access Control — CWE-284 (Improper Access Control), CWE-552 (Files or Directories Accessible to External Parties)	■ Requer ação manual (ver seção 5)
4	Média	Nenhum rate limiting/throttling configurado em nenhum endpoint, incluindo /api/login/. Permite força bruta de senha sem limite de tentativas.	backend/core/settings.py (bloco REST_FRAMEWORK, ausência de DEFAULT_THROTTLE_CLASSES)	A07:2021 Identification & Authentication Failures — CWE-307 (Improper Restriction of Excessive Authentication Attempts)	■ Recomendação (ver seção 5)

#	Severidade	Vulnerabilidade	Local (arquivo:linha)	OWASP / CWE	Status
5	Média	.gitignore incompleto: não cobria backend/media/, arquivos de chave/certificado (.pem, .key, id_rsa), dumps de banco (.sql, .dump, .bak) nem .env.local.	.gitignore	A05:2021 Security Misconfiguration	■ Corrigido
6	Baixa	Arquivo de teste (spidermanwpp.jpg) versionado dentro de backend/media/ — consequência direta do item 5.	backend/media/documentos/spidermanwpp.jpg	A05:2021 Security Misconfiguration	■ Corrigido
7	Informativo	Dependências desatualizadas com CVEs conhecidas: django rest framework 3.17.1 (2 CVEs), python-dotenv 1.1.1 (1 vuln), sqlparse 0.5.5 (5 vulns) no backend; next fixado em 16.2.6 (9 vulnerabilidades, incluindo RCE não-autenticado em Windows e via AVIF na API de otimização de imagem, e SSRF em Server Actions) no frontend.	backend/requirements.txt, frontend/package.json	A06:2021 Vulnerable and Outdated Components	■ Corrigido

Pontos fortes confirmados (não são achados, são verificações positivas)

- **Isolamento multi-tenant (BOLA/IDOR):** `EscritorioScopedMixin` (`backend/advocacia/mixins.py`) filtra corretamente `get_queryset()` de todo `ModelViewSet` pelo escritorio do usuário autenticado (direto ou via `processo__escritorio` para `Movimentacao/Documento/Agenda`). Os endpoints de relatório (`RelatorioClienteView`, `RelatorioProcessoView` e variantes de e-mail/CSV em `backend/advocacia/views.py`) também escopam corretamente seus `.get(id=..., escritorio=usuario.escritorio)`. Não foi encontrado nenhum caminho para um usuário de um escritório acessar dados de outro escritório adivinhando IDs.
- **CORS:** `CORS_ALLOWED_ORIGINS = ["http://localhost:3000"]` — allowlist específica, não wildcard *.
- **Sem injeção de SQL/comando:** nenhum uso de `.raw()`, `cursor.execute()`, `.extra()`, `subprocess`, `os.system`, `eval`, `exec`, `pickle` ou `yaml.load()` em todo o backend — a ORM do Django é usada de

ponta a ponta, eliminando a superfície de SQL injection.

- **XSS no frontend:** único uso de `dangerouslySetInnerHTML` (`frontend/src/app/layout.js`) é uma string estática sem interpolação de dados do usuário — não é uma vulnerabilidade.
-

4. Resumo da Limpeza (correções aplicadas nesta auditoria)

1. `backend/core/settings.py` — `SECRET_KEY`, `DEBUG` e `ALLOWED_HOSTS` passaram a vir de variáveis de ambiente (`DJANGO_SECRET_KEY`, `DJANGO_DEBUG`, `DJANGO_ALLOWED_HOSTS`). Em produção (`DJANGO_DEBUG=False`), o Django agora **recusa subir** sem uma `DJANGO_SECRET_KEY` real definida — elimina a chave fixa exposta permanentemente no histórico do GitHub. Em desenvolvimento local, continua funcionando sem configuração extra (mantém a chave de rascunho só localmente).

2. `backend/advocacia/models.py` — os três campos de upload de documento (`Usuario.documento_identidade`, `Cliente.documento_identidade`, `Documento.arquivo`) agora só aceitam `pdf`, `jpg`, `jpeg`, `png`, `doc`, `docx` via `FileExtensionValidator`. Os campos `ImageField` (foto) não precisaram de mudança — já se autovalidam como imagem real via `Pillow`.

3. `backend/advocacia/serializers.py` — os dois serializers que declaravam `documento_identidade` explicitamente (`AdvogadoRegistroSerializer`, `AdvogadoSerializer`) não herdavam automaticamente o validador do model (comportamento do DRF: só campos auto-gerados herdam `validators` do model). Corrigido adicionando o mesmo validador manualmente nos dois.

4. Migração

`backend/advocacia/migrations/0006_alter_cliente_documento_identidade_and_more.py` — gerada para registrar os novos validadores nos três campos.

5. **Dependências do backend** (`requirements.txt`): `djangorestframework 3.17.1→3.17.2`, `python-dotenv 1.1.1→1.2.2`, `sqlparse 0.5.5→0.6.0`. `pip-audit` confirma **0 vulnerabilidades conhecidas** após a correção (era 8 CVEs em 3 pacotes).

6. **Dependências do frontend** (`package.json`): `next 16.2.6→16.3.4` (estava fixado sem `^`, por isso `npm audit fix` sozinho não corrigia), `eslint-config-next` atualizado junto. `npm audit` confirma **0 vulnerabilidades** após a correção (era 9, incluindo RCE não-autenticado e SSRF).

7. `.gitignore` — adicionado `.env.local`, `*.pem`, `*.key`, `id_rsa*`, `id_dsa*`, `credentials.json`, `*.sql`, `*.dump`, `*.bak`, `*.log`, `backend/media/`, `backend/venv/`, `backend/.venv/`, `frontend/coverage/`.

8. `backend/media/documentos/spidermanwpp.jpg` — removido do controle de versão (`git rm --cached`) agora que `backend/media/` está no `.gitignore`. O arquivo continua no disco local, só não é mais versionado.

9. `backend/.env.example` — documentadas as três novas variáveis (`DJANGO_SECRET_KEY`, `DJANGO_DEBUG`, `DJANGO_ALLOWED_HOSTS`), incluindo o comando para gerar uma `SECRET_KEY` nova.

Validação pós-correção

- Backend: `python manage.py check` → sem erros. `pip-audit -r requirements.txt` → "No known vulnerabilities found". *(A suíte de testes `manage.py test` não pôde ser executada neste ambiente de auditoria por falta de conectividade com o banco Postgres remoto (Supabase) — recomenda-se rodá-la localmente antes do deploy, o que os autores já fazem rotineiramente neste projeto.)*
 - Frontend: `npm audit` → 0 vulnerabilidades. `npm test` → 47/47 testes passando. Build (`next build`) validado anteriormente na mesma versão do Next.js (16.3.4).
-

5. Plano de Resposta a Incidentes e Mitigação Futura

5.1. Rotação de chaves (recomendado mesmo sem evidência de exploração)

Como a `SECRET_KEY` antiga `(django-insecure-=eb#qqbxlkt$51anhk95b6t0d0b0fa-29kugu#7^8+axaztbv+)` ficou publicamente visível no histórico do repositório por todo o desenvolvimento, ela deve ser tratada como **permanentemente comprometida**, mesmo não havendo evidência de exploração ativa:

1. Gerar uma nova chave: `python -c "from django.core.management.utils import get_random_secret_key; print(get_random_secret_key())"`.
2. Definir como `DJANGO_SECRET_KEY` nas variáveis de ambiente de produção (nunca no código).
3. Trocar a chave invalida **todos os tokens JWT já emitidos** (todos os usuários precisarão logar novamente) — efeito colateral esperado e desejado.
4. Se o projeto algum dia rodar com dados reais usando a chave antiga, considerar essa janela de tempo como potencialmente exposta e avisar os escritórios cadastrados.

Não há evidência de que outras credenciais (Supabase, OpenAI, e-mail) tenham vazado — elas nunca estiveram no código, sempre vieram de `.env` não versionado. Ainda assim, como boa prática por estarem em texto plano em `.env` local, recomenda-se rotacioná-las periodicamente e ao trocar de membro da equipe com acesso à máquina de desenvolvimento.

5.2. Ações pendentes que exigem mudança arquitetural (não aplicadas nesta auditoria)

a) Servir documentos por um proxy autenticado, não por URL pública direta Hoje, em modo `DEBUG=True`, qualquer arquivo em `backend/media/` é acessível por quem souber a URL. A correção completa envolve:

- Criar uma `APIView` autenticada (ex.: `GET /api/documentos/<id>/download/`) que verifica `IsAuthenticated` + escopo do escritório do usuário antes de servir o arquivo (reaproveitando o padrão já usado em `RelatorioClienteView/RelatorioProcessoView`).
- Atualizar os serializers para expor essa URL autenticada em vez do caminho direto de `MEDIA_URL`.
- Atualizar o frontend para usar essa nova URL (incluindo o header `Authorization`).
- Como bônus de defesa em profundidade, randomizar o nome dos arquivos no upload (`upload_to` customizado com `uuid4()`), para que mesmo um acesso indevido não consiga adivinhar nomes de arquivo.

Não foi aplicado nesta auditoria por ser um refactor maior (toca serializers, views, frontend e precisa de testes de regressão dedicados) — reduzir o risco imediato mais seguro, sem essa mudança maior, já foi feito: **configurar `DJANGO_DEBUG=False` em produção já desativa completamente essa rota de mídia estática** (o `if settings.DEBUG`: em `backend/core/urls.py:37` não adiciona as URLs de mídia). Ou seja, a correção do item 1 (forçar configuração explícita de produção) já fecha a maior parte do risco prático — mas o proxy autenticado continua recomendado como defesa em profundidade, pois depender só da flag `DEBUG` é frágil (basta alguém esquecer de configurar `DJANGO_DEBUG=False` no deploy).

b) Rate limiting / throttling Adicionar `django-ratelimit` ou o throttling nativo do DRF (`DEFAULT_THROTTLE_CLASSES` + `DEFAULT_THROTTLE_RATES`) no `LoginView`, especialmente por IP e por e-mail tentado. Não aplicado nesta auditoria porque exige decidir um backend de cache (Redis/memcached/local) — decisão de infraestrutura fora do escopo de uma correção pontual.

5.3. Prevenção de vazamentos futuros

Como o histórico está limpo hoje, a prioridade é **manter** esse estado:

1. **Pre-commit hook de detecção de segredos** — instalar `detect-secrets` ou `trufflehog` como hook local (pre-commit) para bloquear commits que contenham padrões de chave/token antes mesmo do `git push`.

```
pip install detect-secrets pre-commit
detect-secrets scan > .secrets.baseline
```

1. **Nunca reduzir o `.gitignore` sem revisão** — o incidente do `spidermanwpp.jpg` mostra como um diretório de upload sem `.gitignore` vira um vazamento de PII em produção.
 2. **Revisão de PR:** antes de aprovar qualquer PR que mexa em `settings.py`, `.env.example` ou em qualquer `FileField/upload`, checar explicitamente se não há segredo em texto plano e se validações de tipo/tamanho continuam presentes.
 3. **Se, no futuro, um segredo real for commitado por engano:** não basta um novo commit apagando o valor — o segredo continua recuperável no histórico. Nesse caso, é necessário reescrever o histórico com [git filter-repo](#) (preferível) ou [BFG Repo-Cleaner](#), seguido de rotação **imediata** da credencial vazada (a reescrita de histórico sozinha não invalida a credencial, só remove a evidência) e de um `force-push` coordenado com todos os colaboradores (que precisarão re-clonar o repositório). Esta auditoria não encontrou necessidade de executar esse procedimento — é documentado aqui apenas como plano de contingência para o futuro.
-

6. Conclusão

O repositório está limpo de vazamentos reais de segredos. A vulnerabilidade mais grave encontrada — a `SECRET_KEY` fixa no código, que também funcionava (sem configuração explícita) como chave de assinatura dos tokens JWT do sistema — foi corrigida nesta auditoria, e representava o maior risco real: `bypass` completo de autenticação entre escritórios. As dependências vulneráveis do backend e do frontend foram todas atualizadas e confirmadas em zero vulnerabilidades conhecidas pelas próprias ferramentas oficiais do ecossistema (`pip-audit`, `npm audit`). As duas recomendações restantes (`proxy` de download autenticado e `rate limiting` de login) exigem mudanças arquiteturais maiores e foram documentadas para implementação futura, já com boa parte do risco prático mitigado indiretamente pela correção do item 1.